

Erweiterter Workflow zur Umwandlung von PDF-Fragen in Moodle-XML für diverse Fragetypen

Kontext und Anforderungen

Der Benutzer möchte PDF-Dateien, die sowohl maschinenlesbaren Text als auch gescannte Seiten enthalten können, in eine Moodle-XML-Datei konvertieren. Die PDFs können relevante oder dekorative Abbildungen enthalten. Die gewünschten Fragetypen umfassen neben Standardtypen wie Multiple Choice, Wahr/Falsch, Kurzantwort und Matching auch Kprim-, MTF- (Multiple True/False-), Wordselect- (Wortauswahl-), Zeichnungs- und Lückentextfragen (Gap-Fill/Cloze). Alle konvertierten Fragen sollen in einer einzigen XML-Datei gespeichert und zum Download bereitgestellt werden.

Erweiterungen gegenüber dem Basis-Workflow

1. Umgang mit gescannten PDFs und Bildern

1. **OCR-Integration:** Wenn `pdfplumber` nur wenig oder gar keinen Text extrahiert, wird ein OCR-Verfahren ausgelöst. Mit Bibliotheken wie `pytesseract` und `pdf2image` lassen sich Seiten in Bilddateien umwandeln und dann mittels OCR als Text auslesen. Dies ermöglicht die Verarbeitung gescannter Dokumente.
2. **Bildextraktion:** `pdfplumber` bietet `page.images`, um eingebettete Bilder zu extrahieren. Jedes extrahierte Bild wird als PNG im Speicher gespeichert und in Base64 kodiert. Die Kodierung kann entweder in das `<questiontext>` (als `img`-Tag mit `data:image/png;base64,...`) eingebettet oder im `<image>`-Tag mit `<image_base64>` gespeichert werden ¹.
3. **Relevanzbewertung:** Da nicht automatisch ersichtlich ist, ob eine Abbildung relevant oder dekorativ ist, bietet das Skript folgende Optionen:
4. **Vorschau:** Nach dem Extrahieren der Bilder zeigt das Skript dem Benutzer eine kleine Vorschau (z. B. als ASCII-Thumbnail oder als separate Bilddatei) und fragt, ob die Abbildung in den Fragetext aufgenommen werden soll.
5. **Metadaten:** Für jede Abbildung werden Seiten- und Positionsinformationen angezeigt (Seite x, Größe, etc.), sodass der Benutzer besser entscheiden kann.
6. **Speicherung:** Gewünschte Bilder werden in den entsprechenden Fragenblock aufgenommen und mit Base64 kodiert. Dekorative Bilder werden verworfen.

2. Erweiterte Fragetypen und Heuristiken

Die folgenden heuristischen Regeln dienen zur vorläufigen Klassifikation. Da die Struktur von PDF-Dateien stark variiert, sollte der Benutzer bei unklaren Fällen den Fragetyp manuell wählen.

Frage­typ	Heuristische Erkennung	Empfohlene XML-Struktur	Besondere Felder
Kprim	Der Fragetext enthält vier Aussagen (oft nummeriert A-D oder 1-4), zu denen jeweils „Richtig/Falsch“ gewählt werden muss.	<pre><question type="kprime"> gefolgt von <scoringmethod> , <shuffleanswers> , <numberofrows>4</ numberofrows> , <numberofcolumns>2</ numberofcolumns> , <row> -Tags mit <optiontext> für jede Aussage und <column> -Tags für die Antwortoptionen „True/ False“. Gewichtungen werden über <weight rownumber="..." columnnumber="..."><value>... </value></weight> gesetzt 2 .</pre>	<pre>scoringmethod</pre> (z. B. <code>kprime</code> , <code>kprimeonezero</code> oder <code>subpoints</code> ³) definiert das Bewertungsschema; <code>numberofrows</code> entspricht der Anzahl Aussagen (meist vier), <code>numberofcolumns</code> ist üblicherweise 2 (True/False).
MTF (Multiple True/False)	Mehrere Aussagen (mehr als vier) müssen jeweils mit „Richtig/Falsch“ bewertet werden.	<pre><question type="mtf"> mit denselben Unterelementen wie kprime, aber numberofrows beliebig und answernumbering (z. B. abc) sowie optional deduction zur Festlegung von Punktabzügen 4 .</pre>	<pre>scoringmethod</pre> (z. B. <code>subpoints</code> oder <code>mtf1/0</code> ⁵), <code>deduction</code> für negative Punkte.
Wordselect (Wortauswahl)	Eine längere Passage, in der bestimmte Wörter markiert werden müssen. Der Fragetext enthält häufig spitze Klammern oder geschweifte Klammern (<code>{wort}</code>) um die richtigen Wörter ⁶ .	<pre><question type="wordselect"> mit den üblichen Grundelementen und zusätzlichen Feldern <delimitchars> und <wordpenalty> 7 . Im Fragetext werden die korrekten Wörter in den angegebenen Delimiter-Zeichen (z. B. { und }) eingeschlossen.</pre>	<pre>delimitchars</pre> bestimmt die verwendeten Trennzeichen (Standard: <code>[]</code>), <code>wordpenalty</code> gibt Punktabzüge für falsche Auswahl an.

Frage­typ	Heuristische Erkennung	Empfohlene XML-Struktur	Besondere Felder
Freehand Drawing	Die Aufgabe fordert dazu auf, etwas zu zeichnen oder ein markiertes Element in einem Bild zu erstellen; es gibt häufig eine Hintergrundgrafik.	<code><question type="freehanddrawing"></code> gefolgt von <code><threshold></code> und <code><radius></code> , die Toleranzwerte für die Auswertung definieren, einem <code><bgimage></code> mit <code><filename></code> und <code><dataURL></code> (Base64 der Hintergrundgrafik) und <code><answer></code> -Tags mit den Koordinaten der korrekten Zeichnung ⁸ .	<code>threshold</code> und <code>radius</code> bestimmen die zulässige Abweichung; der <code><bgimage></code> -Tag enthält die Bilddaten ⁸ .
Lückentext / Cloze	Der Fragetext enthält Lücken (z. B. Unterstriche ___ oder Platzhalter []). Die Lösung steht im PDF meist in Klammern oder als separate Antwort.	<code><question type="cloze"></code> oder <code><question type="gapselect"></code> (je nach installierter Plugin-Variante). Innerhalb des <code><questiontext></code> werden Platzhalter mit geschweiften Klammern notiert, etwa <code>{1:SHORTANSWER:=korrekt}</code> , wobei <code>1</code> für die Lücke steht und <code>SHORTANSWER</code> den Lückentyp bestimmt.	Die genaue Syntax hängt vom verwendeten Cloze-Plugin ab. Für einfache Lückentexte kann <code>SHORTANSWER</code> verwendet werden; für Auswahl-Lücken wird <code>MULTICHOICE</code> benutzt.
Essay	Offene Fragen ohne vorgegebene Lösung oder mit langer Antwort.	<code><question type="essay"></code> mit leeren <code><answer></code> -Tags ⁹ .	Optional: <code>responseformat</code> (z. B. <code>editor</code>), <code>responserequired</code> (1/0), <code>responsefieldlines</code> (Größe des Textfeldes).

Fragetyp	Heuristische Erkennung	Empfohlene XML-Struktur	Besondere Felder
Multiple Choice, True/False, Short Answer, Matching	Siehe Basis-Workflow.	Standard-Moodle-Tags wie <code><question type="multichoice"></code> mit <code><answer></code> -Tags und <code>fraction</code> -Attributen ¹⁰ ; <code><question type="truefalse"></code> mit zwei <code><answer></code> -Tags ¹¹ ; <code><question type="shortanswer"></code> ¹² ; <code><question type="matching"></code> mit <code><subquestion></code> für jede Zuordnung ¹³ .	—

3. Interaktive Abfrage bei unklaren Fällen

Da die Heuristiken nicht jeden Fall korrekt erfassen, wird das Skript so angepasst, dass es beim Erkennen dieser Spezialtypen die folgenden Informationen vom Benutzer abfragt:

- **Kprim/MTF:** Anzahl der Aussagen (Zeilen) und die Beschriftung der Antwortspalten („Wahr“/ „Falsch“ oder abweichend). Für jede Aussage wird gefragt, welche Spalte korrekt ist. Außerdem kann das Bewertungsschema (Scoring-Methode) gewählt werden (`kprime` , `kprimeonezero` oder `subpoints`).
- **Wordselect:** Wahl der Delimiter-Zeichen für die Markierung der richtigen Wörter (Standard `{ }`) und ggf. Strafpunkte (`wordpenalty`). Das Skript liest den Fragetext aus und fordert den Benutzer auf, die richtigen Wörter mit den festgelegten Delimitern zu umgeben.
- **Drawing:** Angabe der Schwellenwerte (`threshold` und `radius`), Auswahl eines Hintergrundbildes (entweder aus dem PDF oder einer externen Datei) und Eingabe der korrekten Zeichnung (z. B. als Liste von Koordinaten oder als SVG-Pfad).
- **Lückentext/Cloze:** Definition der Lücken und ihrer korrekten Lösungen. Das Skript zeigt den Satz mit markierten Platzhaltern und fordert zur Eingabe der korrekten Antwort oder Auswahlmöglichkeiten auf.

4. Erweiterte Skript-Struktur (Python)

Das Skript aus dem Basis-Workflow wird erweitert um zusätzliche Fragetypen und OCR-Funktionen. Wichtige Ergänzungen:

1. OCR-Funktion:

```
from pdf2image import convert_from_path
import pytesseract

def ocr_page(page_image):
    # Führt OCR auf einer PIL-Bildseite aus und gibt den extrahierten Text
```

```

zurück.
    return pytesseract.image_to_string(page_image, lang='deu')

def extract_text_with_ocr(pdf_path):
    # Kombiniert pdfplumber und OCR: zunächst Text extrahieren, dann bei
    Bedarf OCR.
    text_blocks = []
    with pdfplumber.open(pdf_path) as pdf:
        for page in pdf.pages:
            text = page.extract_text() or ''
            if len(text.strip()) < 20: # Annahme: zu wenig Text -> OCR nötig
                images = convert_from_path(pdf_path,
first_page=page.page_number, last_page=page.page_number)
                text = ocr_page(images[0])
            text_blocks.append(text)
    return '\n'.join(text_blocks)

```

1. Bildextraktion und Base64-Einbettung:

```

import base64
from io import BytesIO

def extract_images(page):
    images_data = []
    for img in page.images:
        bbox = (img["x0"], img["top"], img["x1"], img["bottom"])
        # Extrahiere das Bild über die bbox
        cropped = page.crop(bbox).to_image(resolution=300)
        buffer = BytesIO()
        cropped.save(buffer, format="PNG")
        encoded = base64.b64encode(buffer.getvalue()).decode('utf-8')
        images_data.append({'data': encoded, 'extension': 'png'})
    return images_data

```

1. Erweiterte Klassifikation:

Zusätzlich zu den heuristischen Regeln für Standardtypen prüft `classify_question` auf Schlüsselwörter und Strukturen, die auf Kprim/MTF, Wordselect, Zeichnen oder Cloze hinweisen. Beispiele:

```

def classify_question(block_text):
    text_lower = block_text.lower()
    # Lückentext: Suche nach langen Unterstrichen oder Platzhaltern
    if re.search(r'_\s*_||\{\s*\}', block_text):
        return 'cloze'
    # Kprim: vier Aussagen mit True/Falsch
    if len(re.findall(r'^\s*[abcd]\)', block_text, re.MULTILINE)) == 4 and

```

```

('richtig' in text_lower or 'wahr' in text_lower):
    return 'kprime'
# MTF: mehrere Aussagen > 4
if len(re.findall(r'^\s*[a-z]\s', block_text, re.MULTILINE)) > 4 and
('richtig' in text_lower or 'wahr' in text_lower):
    return 'mtf'
# Zeichnung: Schlüsselwörter
if any(word in text_lower for word in ['zeichne', 'zeichnen',
'skizziere']):
    return 'freehanddrawing'
# Wordselect: „markiere/markieren“ und längerer Satz
if 'markieren' in text_lower or 'wählen sie' in text_lower:
    return 'wordselect'
# Standardtypen
# ... (wie im Basis-Skript)

```

1. XML-Generator für neue Fragetypen:

Das bisherige `to_xml`-Verfahren wird pro Fragetyp erweitert. Beispiel für Kprim:

```

elif self.qtype == 'kprime':
    # Annahme: self.options enthält eine Liste von Aussagen und
    selbst.qcolumns zwei Spalten ('True','False')
    q = ET.Element('question', type='kprime')
    # Name und Frage analog zu vorher
    ET.SubElement(q, 'scoringmethod').text = self.scoringmethod
    ET.SubElement(q, 'shuffleanswers').text = '1'
    ET.SubElement(q, 'numberofrows').text = str(len(self.options))
    ET.SubElement(q, 'numberofcolumns').text = '2'
    # Zeilen
    for idx, option in enumerate(self.options, start=1):
        row_el = ET.SubElement(q, 'row', number=str(idx))
        opttext = ET.SubElement(row_el, 'optiontext', format='html')
        ET.SubElement(opttext, 'text').text = option['text']
        feedback = ET.SubElement(row_el, 'feedbacktext', format='html')
        ET.SubElement(feedback, 'text').text = option.get('feedback', '')
    # Spalten (Antwortoptionen)
    for col_idx, resp_text in enumerate(['True','False'], start=1):
        col_el = ET.SubElement(q, 'column', number=str(col_idx))
        resp = ET.SubElement(col_el, 'responsetext',
format='moodle_auto_format')
        ET.SubElement(resp, 'text').text = resp_text
    # Gewichtungen (korrekte Antworten)
    for row_idx, option in enumerate(self.options, start=1):
        # option['correct'] ist index der korrekten Spalte (1 oder 2)
        for col_idx in [1, 2]:
            weight_el = ET.SubElement(q, 'weight', rownumber=str(row_idx),
columnnumber=str(col_idx))
            val = '1' if option['correct'] == col_idx else '0'

```

```
val_el = ET.SubElement(weight_el, 'value')
val_el.text = val
```

Für die anderen Spezialtypen werden ähnliche Konstruktionen implementiert, wobei Wordselect die Trennzeichen (`delimitchars`) und `wordpenalty` einfügt, Freehand Drawing das Hintergrundbild (`bgimage`) und die `threshold` / `radius` -Parameter sowie die korrekte Zeichnung (in einem `<answer>` -Tag) definiert, und Cloze/Lückentext in den `<questiontext>` Lücken mittels `SHORTANSWER` , `MULTICHOICE` usw. einbettet.

1. Generierung der gemeinsamen XML-Datei:

Alle Fragen werden in einer einzigen `<quiz>` -Datei gespeichert. Das Skript verwaltet eine Liste von `QuestionBlock` -Objekten, die jeweils den identifizierten Fragetyp, den Fragetext, Bilder und zusätzliche Optionen enthalten. Nach Bearbeitung aller Fragen schreibt das Skript den `<quiz>` -Container mit allen `<question>` -Elementen an eine einzige XML-Datei (UTF-8-kodiert).

Fazit

Der erweiterte Workflow ergänzt den Basis-Ansatz um die Verarbeitung gescannter PDFs via OCR, die Extraktion und Einbettung von Bildern sowie die Unterstützung zusätzlicher Moodle-Fragetypen wie Kprim, MTF, Wordselect, Zeichnen und Lückentext. Für die Spezialtypen wird die XML-Struktur anhand der jeweiligen Plugins (z. B. Kprim/MTF ² ⁴ , Wordselect ⁷ , Freehand Drawing ⁸) nachgebildet. Die heuristischen Erkennungsregeln liefern erste Vorschläge; letztlich sollte der Benutzer die automatisch erkannten Fragetypen und Inhalte prüfen und bei Bedarf manuell korrigieren.

Zusätzliche Optimierungen gemäß weiterer Spezifikationen

Bildoptimierung und Upscaling

Um eingebettete Bilder aus PDF- und DOCX-Dateien webtauglich zu machen, sollte das Skript die Bildgröße reduzieren oder, falls die ursprüngliche Auflösung zu gering ist, mit einem Upscaling-Verfahren verbessern. Moodle-Anleitungen empfehlen, Bilder in Quizfragen vor dem Upload auf eine Breite zwischen 400 – 600 px (oder eine entsprechende Höhe bei Hochformat) zu skalieren ¹⁴ . Für die Archivierung von Quizzes wird zudem geraten, maximale Bilddimensionen festzulegen und die Qualität zu komprimieren, wobei ein Wert von 85 % ein guter Kompromiss zwischen Dateigröße und Qualität ist ¹⁵ . Das Skript verwendet daher standardmäßig eine maximale Kantenlänge von 800 px und optimiert PNG-Dateien; optional kann eine externe Upscaling-Bibliothek (z. B. Upscayl) integriert werden, um niedrige Auflösungen zu verbessern.

Relevanz von Bildern erkennen

Nicht alle Bilder im Ausgangsdokument sind für die Frage relevant. Um Benutzerinteraktionen zu minimieren, erkennt das Skript Schlüsselwörter wie „Abbildung“, „Diagramm“ oder „Bild“ im Fragetext. Sind solche Begriffe vorhanden, werden alle extrahierten Bilder automatisch an den Fragetext angehängt. Andernfalls wird der Benutzer gefragt, ob die Bilder aufgenommen werden sollen. Die Bilder werden als HTML-`<img>` -Tag mit Data-URIs (Base64-kodiert) in den `<questiontext>` eingebettet. So bleiben keine zusätzlichen Dateien übrig und die Bilder werden mit dem Fragetext transportiert.

Unterstützung von DOCX-Dateien

Zusätzlich zu PDF-Dateien verarbeitet der Workflow nun auch Word-Dokumente (`.docx`). Mittels `python-docx` werden die Textinhalte aus den Absätzen extrahiert sowie eingebettete Bilder aus den Beziehungen der DOCX-Datei geladen. Diese Bilder werden analog zu PDF-Bildern optimiert und in Base64-Form in die resultierende XML eingebettet. Benutzer können somit entweder eine PDF- oder eine DOCX-Datei in die Pipeline geben und erhalten am Ende eine Moodle-XML mit allen extrahierten Fragen.

Standardwerte für Punkte

Falls im Ausgangsdokument keine Punktzahl angegeben ist, vergibt das Skript standardmäßig **1,0 Punkt** pro Frage. Während der interaktiven Eingabe kann der Benutzer diesen Wert anpassen. In der XML-Ausgabe wird die Punktzahl über das `<defaultgrade>`-Element festgelegt.

Fehleranalyse und interaktive Korrekturen

Das Skript protokolliert Warnungen, wenn es unklare Strukturen entdeckt (z. B. wenn eine Frage nicht eindeutig einem Fragetyp zugeordnet werden kann). In solchen Fällen wird der Benutzer im Terminal/ Chat aufgefordert, den richtigen Fragetyp zu wählen oder fehlende Informationen zu ergänzen (z. B. korrekte Antworten bei Multiple Choice). So wird sichergestellt, dass trotz heuristischer Erkennung eine saubere Moodle-XML entsteht. Unbehandelte Fehler werden nicht stillschweigend ignoriert, sondern dem Benutzer zur Klärung vorgelegt.

1 9 10 11 12 13 Moodle XML format - MoodleDocs

https://docs.moodle.org/502/en/Moodle_XML_format

2 raw.githubusercontent.com

https://raw.githubusercontent.com/ethz-let/moodle-qtype_kprime/master/questiontype.php

3 Kprime question type - MoodleDocs

https://docs.moodle.org/502/en/Kprime_question_type

4 raw.githubusercontent.com

https://raw.githubusercontent.com/ethz-let/qtype_mtf/master/questiontype.php

5 Multiple True/False Question Type - MoodleDocs

https://docs.moodle.org/502/en/Multiple_True/False_Question_Type

6 Wordselect question type - MoodleDocs

https://docs.moodle.org/502/en/Wordselect_question_type

7 raw.githubusercontent.com

https://raw.githubusercontent.com/marcusgreen/moodle-qtype_wordselect/master/questiontype.php

8 raw.githubusercontent.com

https://raw.githubusercontent.com/marcusgreen/moodle-qtype_freehanddrawing/master/questiontype.php

14 Adding Images or Videos to Exam Questions - 4.5 : IET Knowledge Base

<https://helpdesk.lsua.edu/support/solutions/articles/48000986411-adding-images-or-videos-to-exam-questions-4-5>

15 Image Optimization - Moodle Quiz Archiver Documentation

<https://quizarchiver.gandrass.de/usage/imageoptimization/>